OBSTACLE AVOIDING ROBOT WITH VOICE CONTROL

Video link:1T2Vd43Rnf8QR5yyCprjT6AgFQhGlEyhW

OBJECTIVE:

The objective of this project is to design and implement an intelligent mobile robot that combines **autonomous navigation** with **human-friendly voice interaction**. The robot should be capable of:

1. **Safe Autonomous Navigation**
    - Detecting and avoiding obstacles in real-time using ultrasonic/IR sensors.
    - Ensuring smooth and collision-free movement in dynamic indoor environments such as homes, hospitals, and offices.
2. **Voice-Based Human Interaction**
    - Recognizing and executing user commands through speech, eliminating the need for physical controllers.
    - Providing an intuitive interface for people of all ages, including the elderly and differently-abled.
3. **Integration of Control Systems**
    - Combining obstacle-avoidance algorithms with voice-command overrides to create a hybrid navigation model (autonomous + user-directed).
    - Ensuring seamless communication between the robot's sensors, actuators, and voice recognition module.
4. **Practical Usability**
    - Acting as a low-cost prototype for service robots used in homes, educational institutions, healthcare facilities, and industries.
    - Demonstrating how simple, scalable robotics can solve real-world problems like assistance, monitoring, and automation.
5. **Research & Learning Purpose**
    - Providing a hands-on platform for understanding embedded systems, control algorithms, IoT, and artificial intelligence.

Problem Statement

With the advancement of embedded systems, robotics has become a vital area for automation and control. However, most robots are limited to either manual or automatic operation.
This project addresses that gap by building a dual-mode robot — capable of automatic obstacle avoidance and manual control through voice commands using a Bluetooth interface. The design uses Arduino Uno as the main controller, which processes inputs from an ultrasonic sensor to detect nearby obstacles and voice commands via a Bluetooth module to control motion through an L293D motor driver**.**

Key Contributions

- Implementation of an Arduino-based robot with dual-mode operation.
- Integration of ultrasonic sensing for obstacle avoidance.
- Use of Bluetooth communication for wireless voice control.
- Cost-effective and simple construction using readily available **components**.
- Design and testing of a practical robotic model for academic and research learning.

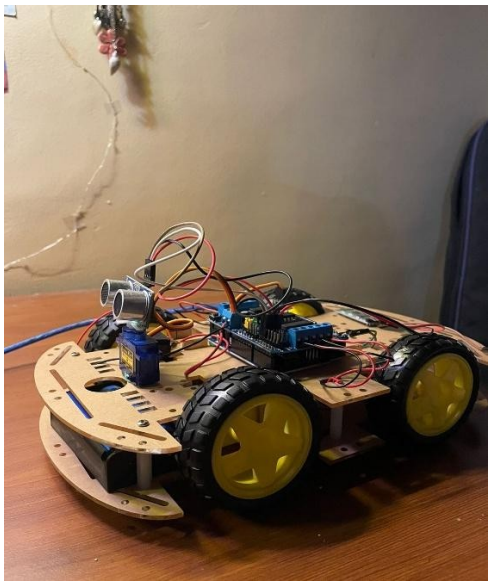
System Model

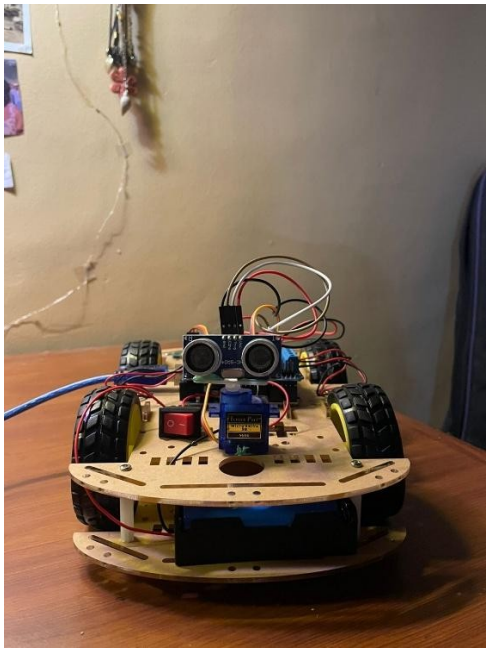
Proposed Methodology

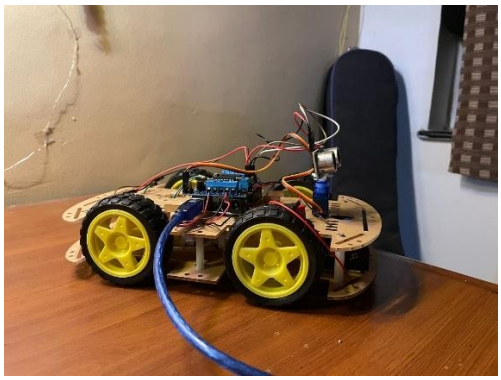
The robot system is designed around the Arduino Uno microcontroller, which serves as the control unit for both autonomous navigation and manual control.

In obstacle avoidance mode, the HC-SR04 ultrasonic sensor continuously measures the distance to nearby objects. When an obstacle is detected within a threshold (around 15 cm), the Arduino stops the robot and redirects its motion.

In voice control mode, commands such as “forward,” “backward,” “left,” “right,” and “stop” are transmitted through the HC-05 Bluetooth module from a smartphone app. The Arduino interprets these commands and drives the motors accordingly.







Communication and Control Flow

- **Autonomous Mode:** Ultrasonic sensor data is continuously read by Arduino. If the measured distance is below the threshold, the robot stops, turns, and moves in a new direction.
- **Voice Control Mode:** Bluetooth module receives commands from a mobile app, sends them via serial communication to the Arduino, which then moves the robot accordingly.
- The servo motor rotates the ultrasonic sensor to detect obstacles in different directions.

RESULTS & DISCUSSIONS

Results

The project was successfully implemented and tested for both obstacle avoidance and voice control functionalities.

- The robot accurately detected obstacles within a range of 2 cm to 100 cm.
- Voice commands were recognized efficiently through the Bluetooth interface.
- The robot maintained stability and smooth operation across multiple surfaces.

Challenges and Improvements

- Limited Bluetooth Range: The HC-05 module operates effectively within 10 meters.
- Power Drain: Li-ion batteries require frequent recharging after extended operation.
- Sensor Limitations: Performance may vary under bright sunlight or uneven surfaces.

SYSTEM DESIGN AND METHODOLOGY

Hardware Components

Component	Function
Arduino UNO	Processes data from sensors and executes control logic
L293D Motor Driver	Drives the DC motors based on Arduino signals
Ultrasonic Sensor (HC-SR04)	Measures distance to detect obstacles
Bluetooth Module (HC-05)	Receives voice commands wirelessly
Servo Motor	Rotates sensor to scan surroundings

Component	Function
Gear Motors (4)	Provide robot movement
Li-ion Batteries	Power source for Arduino and motors
Foam Board & Cardboard	Used for the robot's chassis structure

Circuit Description

- Ultrasonic Sensor (HC-SR04):
  - VCC → 5V, GND → GND, TRIG → D9, ECHO → D10
- Bluetooth Module (HC-05):
  - VCC → 5V, GND → GND, TX → D2 (RX), RX → D3 (TX)
- L293D Motor Driver:
  - IN1 → D4, IN2 → D5, IN3 → D6, IN4 → D7
  - Motor outputs connected to left and right motors
  - 12V → Battery, 5V output → Arduino
- Servo Motor:
  - Signal → D11, VCC → 5V, GND → GND

CONCLUSION

The Obstacle Avoiding Robot with Voice Control was successfully designed, implemented, and tested. The system integrates ultrasonic obstacle detection with Bluetooth-based voice control, providing both autonomous and manual control modes.

This project highlights the effective use of Arduino-based embedded systems**,** sensors**,** and wireless communication for intelligent robotics. It serves as a base model for developing more advanced systems involving IoT, AI, and computer vision in the future.

REFERENCES

1. SriTu Hobby, *"How to Make a Multi-Function Arduino Robot,"* [Online]. Available: https://srituhobby.com/how-to-make-a-multi-function-arduino-robot/
2. Arduino Documentation – *Ultrasonic Sensor HC-SR04 Usage Guide.*
3. HC-05 Bluetooth Module Datasheet.
4. L293D Motor Driver Datasheet.

5. Arduino IDE Reference – *Serial Communication Functions.*

Real time applications:

1. Assistive Robots for Elderly & Differently-Abled

- Voice commands let users who can't operate complex controls interact easily.
- Obstacle avoidance ensures safe indoor navigation without collisions.
- Can be used for fetching small items, reminders, or companionship.

2. Smart Home Service Robots

- A home robot that moves around to deliver items, control appliances, or monitor rooms.
- Voice recognition makes it user-friendly for children and non-technical people.
- Avoidance prevents damage to furniture or pets.

3. Security & Surveillance Robots

- Can patrol halls/offices autonomously while avoiding obstacles.
- Accepts voice overrides ("stop", "come here", "return to base").
- Equipped with cameras/sensors for intrusion detection.

4. Warehouse & Industry Automation

- Small-scale delivery robots in warehouses/factories.
- Workers can control them with voice commands while keeping their hands free.
- Obstacle avoidance ensures safe operation in busy environments**.**

5. Healthcare & Hospital Robots

- Used to deliver medicines, food, or files to patients/doctors.
- Voice interaction allows nurses/doctors to command the robot quickly.
- Safe navigation in crowded hospital corridors.

6. Educational & Research Platforms

- Great project for teaching AI, robotics, and IoT integration.
- Can be extended with machine learning for advanced navigation.

- Often used in robotics labs as a base for experiments.

Code:

```
/*
 * Obstacle-Avoiding, Bluetooth-Controlled and Voice-Controlled Robot Car
 *
 * Hardware:
 * - Arduino Uno
 * - L293D motor shield compatible with AFMotor library
 * - HC-SR04 ultrasonic sensor
 * - Servo motor
 * - Bluetooth module (e.g., HC-05), if using Bluetooth control
 *
 * Ultrasonic sensor:
 *   Echo -> A0
 *   Trig -> A1
 *
 * Servo signal -> Digital pin 10
 *
 * IMPORTANT:
 * The motor shield and servo pin assignments must be compatible
 * with your actual hardware.
 */

#include <Servo.h>
#include <AFMotor.h>

#define ECHO_PIN A0
#define TRIG_PIN A1
#define SERVO_PIN 10
```

```
#define MOTOR_SPEED 170
#define SERVO_CENTER 103

// Select ONE operating mode:
// 1 = Obstacle avoidance
// 2 = Bluetooth control
// 3 = Voice control (serial commands)
#define CONTROL_MODE 1

#define OBSTACLE_DISTANCE_CM 12
#define ULTRASONIC_TIMEOUT_US 30000UL

char value = '\0';
int distance = 0;

Servo servo;

AF_DCMotor M1(1);
AF_DCMotor M2(2);
AF_DCMotor M3(3);
AF_DCMotor M4(4);

void forward();
void backward();
void left();
void right();
void Stop();

int ultrasonic();
int leftsee();
```

```
int rightsee();

void Obstacle();
void Bluetoothcontrol();
void voicecontrol();

void setup() {
  Serial.begin(9600);

  pinMode(TRIG_PIN, OUTPUT);
  pinMode(ECHO_PIN, INPUT);
  digitalWrite(TRIG_PIN, LOW);

  servo.attach(SERVO_PIN);
  servo.write(SERVO_CENTER);
  delay(500);

  M1.setSpeed(MOTOR_SPEED);
  M2.setSpeed(MOTOR_SPEED);
  M3.setSpeed(MOTOR_SPEED);
  M4.setSpeed(MOTOR_SPEED);

  Stop();

  Serial.println("Robot car initialized.");
}

void loop() {
#if CONTROL_MODE == 1
  Obstacle();
```

```
#elif CONTROL_MODE == 2
  Bluetoothcontrol();
#elif CONTROL_MODE == 3
  voicecontrol();
#else
  Stop();
#endif
}

// Bluetooth commands:
// F = forward, B = backward, L = left,
// R = right, S = stop
void Bluetoothcontrol() {
  if (Serial.available() > 0) {
    value = Serial.read();

    switch (value) {
      case 'F':
        forward();
        break;
      case 'B':
        backward();
        break;
      case 'L':
        left();
        break;
      case 'R':
        right();
        break;
      case 'S':
```

```
      Stop();
      break;
    default:
      break;
   }
  }
}

// Obstacle avoidance
void Obstacle() {
  distance = ultrasonic();

  if (distance > 0 && distance <= OBSTACLE_DISTANCE_CM) {
   Stop();
   delay(150);

   backward();
   delay(300);
   Stop();
   delay(150);

   int leftDistance = leftsee();
   int rightDistance = rightsee();

   servo.write(SERVO_CENTER);
   delay(200);

   if (leftDistance == 0 && rightDistance == 0) {
    backward();
    delay(300);
```

```
    } else if (leftDistance > rightDistance) {
      left();
      delay(450);
    } else {
      right();
      delay(450);
    }

    Stop();
    delay(100);
  } else if (distance == 0) {
    // No valid echo: stop rather than assume the path is clear.
    Stop();
  } else {
    forward();
  }
}

// Voice-control commands received over Serial.
// ^ = forward, - = backward, < = left,
// > = right, * = stop
void voicecontrol() {
  if (Serial.available() > 0) {
    value = Serial.read();

    switch (value) {
      case '^':
        forward();
        break;
      case '-':
```

```
      backward();
      break;
    case '<':
      left();
      delay(400);
      Stop();
      break;
    case '>':
      right();
      delay(400);
      Stop();
      break;
    case '*':
      Stop();
      break;
    default:
      break;
   }
  }
}

// Returns distance in centimeters.
// Returns 0 if no valid echo is received.
int ultrasonic() {
  digitalWrite(TRIG_PIN, LOW);
  delayMicroseconds(2);

  digitalWrite(TRIG_PIN, HIGH);
  delayMicroseconds(10);
  digitalWrite(TRIG_PIN, LOW);
```

```
  unsigned long duration =
      pulseIn(ECHO_PIN, HIGH, ULTRASONIC_TIMEOUT_US);

  if (duration == 0) {
    return 0;
  }

  return (int)(duration / 58UL);
}

// Motor movement functions
void forward() {
  M1.run(FORWARD);
  M2.run(FORWARD);
  M3.run(FORWARD);
  M4.run(FORWARD);
}

void backward() {
  M1.run(BACKWARD);
  M2.run(BACKWARD);
  M3.run(BACKWARD);
  M4.run(BACKWARD);
}

void right() {
  M1.run(BACKWARD);
  M2.run(BACKWARD);
  M3.run(FORWARD);
```

```
  M4.run(FORWARD);
}

void left() {
  M1.run(FORWARD);
  M2.run(FORWARD);
  M3.run(BACKWARD);
  M4.run(BACKWARD);
}

void Stop() {
  M1.run(RELEASE);
  M2.run(RELEASE);
  M3.run(RELEASE);
  M4.run(RELEASE);
}

// Measure distance to the left
int leftsee() {
  servo.write(180);
  delay(400);

  int d = ultrasonic();

  return d;
}

// Measure distance to the right
int rightsee() {
  servo.write(20);
```

```
  delay(400);


  int d = ultrasonic();


  return d;
}
```